

Report tecnico del lavoro svolto

Pipeline finale 'make coffee': percezione, spatial-prior gate, VLM e integrazione CycloneDDS - 2026-06-04

0. Come leggere questo documento

Questo report e' pensato per essere riletto a distanza di tempo e capire OGNI dettaglio di cio' che e' stato affrontato e risolto. Per ogni argomento trovi: (1) il problema concreto, (2) la causa tecnica, (3) la soluzione applicata, (4) le variabili e i file coinvolti, (5) i modelli usati e il perche', (6) come il pezzo si incastra nella pipeline finale.

Convenzioni: 'mu' indica il vettore medio del prior gaussiano, 'sigma' la matrice di covarianza, '->' indica un flusso o una trasformazione. I percorsi file sono relativi alla radice del repo indicato.

1. Contesto e obiettivo della pipeline finale

Obiettivo: eseguire in sicurezza il task 'make coffee' sul robot Panda combinando quattro componenti che si controllano a vicenda invece di fidarsi ciecamente di una sola fonte:

- Percezione RGB-D (nodo ROS 2): rileva gli oggetti e ne stima la posa 3D nel frame base_link. E' l'UNICA fonte di verita' sulla posizione degli oggetti.
- Spatial-prior gate (deterministico): PRIMA di lanciare una skill, verifica che la posa osservata dell'oggetto sia compatibile con la distribuzione delle pose viste in addestramento. Se non lo e', blocca o segnala.
- Skill ACT (policy neurale): esegue il gesto vero e proprio (prendere la capsula e inserirla, chiudere la macchina).
- VLM verifier (Qwen3-VL): DOPO l'esecuzione valuta semanticamente la scena per dire se lo stato finale e' corretto.

Il principio di sicurezza e' un fail-safe ASIMMETRICO: il gate, in caso di dubbio, si astiene/blocca (conservativo, non muove il robot a rischio); il VLM, in caso di dubbio, lascia proseguire (RUNNING) per non interrompere inutilmente. Le due barriere coprono errori diversi (geometrico vs semantico).

2. Problema #1 (il piu' critico): il gate si astiene SEMPRE

2.1 Sintomo

Con la percezione reale collegata, lo spatial-prior gate restituiva sempre ABSTAIN con reason=no_translation. In pratica la barriera geometrica era morta: non passava (PASS) ne' bloccava (FAIL), si limitava ad astenersi, quindi non proteggeva nulla.

2.2 Causa tecnica

Il nodo di percezione serializza la posa dell'oggetto con la traslazione come DIZIONARIO {x, y, z} (vedi build_object_pose_fact). La funzione parse_object_pose_json in spatial_prior_gate.py, invece, accettava la traslazione SOLO come LISTA [x, y, z]. Ricevendo un dict, non trovava una lista valida -> translation=None -> ObservedPose con error='no_translation' -> il gate non aveva una posa da valutare -> ABSTAIN permanente.

2.3 Soluzione

Ho reso il parser tollerante a ENTRAMBI i formati. File: src/lerobot_bt_python/spatial_prior_gate.py (repo lerobot).

```
raw = pose.get("translation")
```

```

if isinstance(raw, Mapping):          # formato percezione reale {x,y,z}
    translation = [float(raw["x"]), float(raw["y"]), float(raw["z"])]
elif isinstance(raw, (list, tuple)):  # formato legacy [x, y, z]
    translation = [float(v) for v in raw]
else:
    translation = None

```

Ho aggiunto 'Mapping' agli import di typing. Risultato: con la percezione reale il gate ora estrae correttamente la posa e produce PASS/FAIL invece di astenersi.

2.4 Test aggiunti

File: src/lerobot_bt_python/test_spatial_prior.py. Tre nuovi test: (a) il parser accetta la traslazione come dict; (b) il gate da PASS su un payload reale in base_link vicino a mu; (c) il gate si astiene se il payload e' in frame camera (non base_link). Totale suite: 30 test, tutti verdi.

3. Lo spatial-prior gate in dettaglio

3.1 Modello statistico

Il prior e' una gaussiana 3D sulla posizione dell'oggetto al momento della presa. Si verifica la compatibilita' con la distanza di Mahalanobis:

$$d(x) = \sqrt{(x - \mu)^T * \Sigma^{-1} * (x - \mu)}$$

Se $d(x) \leq \text{soglia} \rightarrow \text{PASS}$ (la posa e' plausibile). Se $d(x) > \text{soglia} \rightarrow \text{FAIL}$ (posa anomala). Se manca la posa o il frame non e' confrontabile $\rightarrow \text{ABSTAIN}$.

3.2 Variabili del prior 'put_coffee.json' (oggetto: coffee_capsule)

- frame_id = base_link (le pose vanno confrontate in questo frame).
- mu = [0.68432, -0.26041, 0.15019] m (media della posizione end-effector all'istante di presa, usata come PROXY della posa oggetto).
- mahalanobis_threshold = 3.36821 (soglia a confidence_level 0.99 su 3 gradi di liberta').
- sigma = matrice 3x3 di covarianza; translation_std_m ~ [0.026, 0.028, 0.005] m (la profondita' z e' molto piu' stretta).
- n_demos = 50, proxy = grasp_ee_position, min_pose_confidence = 0.0.

Nota importante: mu/sigma sono fittati sulla posizione dell'end-effector, NON direttamente sul centroide dell'oggetto visto dalla percezione. Esiste quindi un offset fisso di presa; per questo il gate parte in modalita' SHADOW e va calibrato prima di passare a ENFORCE.

3.3 Modalita' operative e logging

- shadow: il gate calcola e logga il verdetto ma NON blocca la skill (serve a raccogliere dati e calibrare). E' la modalita' attuale in make_coffee_executor.yaml.
- enforce: il gate blocca davvero la skill su FAIL.

Ogni decisione emette una riga di log greppabile, ad esempio:

```

event=spatial_prior_gate mode=shadow skill=pick_and_insert_capsule \
  object=coffee_capsule status=PASS reason=within_threshold \
  distance=1.83 threshold=3.368 frame=base_link n_demos=50

```

4. Problema #2: calibrazione camera->base (camera_static_tf_map)

4.1 Perche' serve

La percezione vede gli oggetti nel frame della camera. Per confrontarli col prior (in base_link) serve la trasformazione statica camera->base. Senza una calibrazione reale, qualunque posa risulterebbe sistematicamente sbagliata.

4.2 Cosa ho aggiunto nel config

In src/lerobot_bt_python/make_coffee_executor.yaml ho aggiunto, accanto a camera_frame_id_map (wrist->panda_wrist_camera, left->panda_front_camera), un blocco placeholder COMMENTATO 'camera_static_tf_map' con valori a zero e il commento '# REPLACE WITH REAL CALIBRATION', così la procedura è evidente e non si rischia di lasciare valori finti attivi.

4.3 Helper di calibrazione (nuovo script)

File nuovo: scripts/calibrate_camera_extrinsics.py (repo panda_live_viewer). Risolve la trasformazione rigida camera->base da una lista di corrispondenze punto-punto.

- solve_rigid_transform(camera, base): algoritmo di Kabsch/Umeyama; restituisce R (rotazione 3x3), t (traslazione) e rms (errore residuo).
- rotation_matrix_to_quaternion_xyzw(matrix): converte R in quaternion [x, y, z, w] (formato usato dal TF).
- _format_yaml_block(...): stampa direttamente il blocco 'camera_static_tf_map:' pronto da incollare nel config.
- main(): legge un JSON {camera_name, parent_frame_id, child_frame_id, correspondences:[{camera:[x,y,z], base:[X,Y,Z]}]}, stampa l'RMS e avvisa se supera 20 mm (calibrazione poco affidabile). Output opzionale con --output.

Verifica: su una trasformazione rigida esatta il solver restituisce RMS=0 e recupera R, t e quaternion esattamente.

5. Problema #3: doppia sorgente delle estrinseche (TF ridondante)

Il nodo di percezione aveva un proprio broadcaster delle estrinseche (_init_static_extrinsics + parametro camera_extrinsics_json). Coesistendo con camera_static_tf_map del config lerobot, c'erano DUE fonti per la stessa trasformazione base->camera, con rischio di conflitto.

Soluzione: ho rimosso da perception/node.py la chiamata e il metodo _init_static_extrinsics e il parametro camera_extrinsics_json. Ora la SINGOLA fonte di verità per l'estrinseca base->camera è camera_static_tf_map nel config lerobot.

6. Problema #4: arricchire il VLM con le scene-facts (one-way)

6.1 Obiettivo

Dare al VLM verifiers il contesto numerico della percezione (oggetti presenti e relative pose) per giudizi più informati, MANTENENDO il flusso a senso unico: la percezione informa il VLM, ma il VLM non modifica mai la percezione (nessun anello di retroazione che falsi la fonte di verità).

6.2 Implementazione

- vlm_live/prompt.py: nuova format_scene_context(scene_facts_json) che rende gli oggetti presenti come, ad es., '- coffee_capsule: [0.684, -0.260, 0.150] m in base_link, confidence 0.62'. Restituisce stringa vuota se non ci sono fatti utili.
- build_prompt aggiunge il blocco 'Perception scene facts (...)' SOLO se request contiene 'scene_context'; senza, nessuna regressione.
- vlm_live/node.py: importa format_scene_context; in _run_vlm calcola scene_context dai fatti più

recenti e lo inietta nella request prima di build_prompt (solo se non già presente). _on_scene_facts fa da cache dell'ultimo JSON, _get_latest_scene_facts_json lo espone.

Test: tests/test_vlm_status_protocol.py, classe SceneContextEnrichmentTests (5 test). Totale suite: 15 test verdi.

7. Smoke test offline dello spatial-prior gate

File nuovo: scripts/smoke_spatial_prior_gate.py (repo lerobot). Costruisce un vero SpatialPriorGate da put_coffee.json in modalita' ENFORCE e gli passa tre payload sintetici con traslazione in formato dict (come la percezione reale):

- PASS: oggetto in base_link vicino a mu ([0.684, -0.260, 0.150]).
- FAIL: oggetto in base_link lontano ([1.50, 0.90, 0.80]).
- ABSTAIN: oggetto nel frame panda_front_camera (non confrontabile).

Stampa righe greppabili 'event=spatial_prior_gate ...' e infine 'SMOKE TEST PASSED/FAILED' con exit 0/1. Eseguito: i tre verdetti escono corretti. ATTENZIONE: va lanciato con l'ambiente conda 'lerobot' attivo (vedi sez. 11).

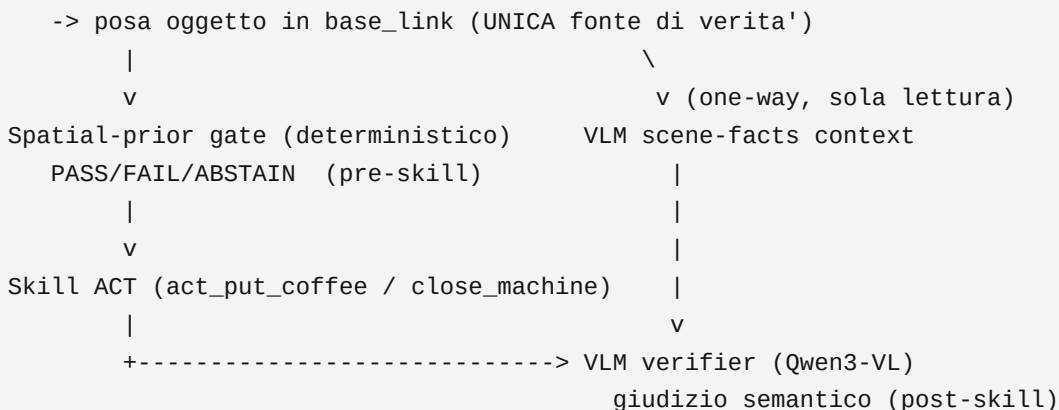
8. Modelli usati e perche'

- Skill ACT 'pick_and_insert_capsule' -> HSP-IIT/act_put_coffee. Policy neurale ACT (action chunking) addestrata per prendere la capsula e inserirla. Skill 'close_machine' -> Squitieri/act_close_machine.
- Percezione (segmentazione zero-shot) -> OWL-ViT (google/owlvit-base-patch32) come backend di default; rileva oggetti da alias in linguaggio naturale (es. 'coffee capsule') e li converte in maschere. Lo strumento diagnostico wrist_snapshot usa OWLv2 (google/owlv2-base-patch16-ensemble), piu' accurato per le ispezioni puntuali.
- VLM verifier -> Qwen3-VL-32B-Instruct. Modello vision-language che valuta semanticamente la scena dopo l'esecuzione. Scelto per la capacita' di ragionamento multimodale; in caso di incertezza lascia proseguire (RUNNING) per non bloccare inutilmente.
- Prior spaziale gaussiano (mu, sigma) fittato da 50 dimostrazioni con fit_spatial_prior.py. Non e' una rete neurale: e' un modello statistico leggero e interpretabile, ideale come barriera deterministica pre-skill.

9. Come tutto si integra nella pipeline finale

Flusso end-to-end del task 'make coffee':

Percezione RGB-D



Punti chiave dell'integrazione: (1) la percezione e' l'unica fonte di posa; (2) il gate e' una barriera

geometrica PRIMA della skill; (3) la skill ACT esegue; (4) il VLM giudica DOPO. Le scene-facts viaggiano solo dalla percezione al VLM, mai all'indietro. Il fail-safe e' asimmetrico: il gate si astiene/blocca nel dubbio, il VLM lascia proseguire nel dubbio.

10. Integrazione automatica di CycloneDDS (locale + server)

10.1 Problema

Ad ogni avvio bisognava esportare a mano quattro impostazioni ROS 2: ROS_DOMAIN_ID, RMW_IMPLEMENTATION, CYCLONEDDS_URI e fare 'unset ROS_LOCALHOST_ONLY'. Operazione ripetitiva e facile da dimenticare, sia in locale sia sul server.

10.2 Soluzione: un unico file sorgenziabile per repo

Ho creato 'ros_env.sh' nella radice di ENTRAMBI i repo (lerobot e panda_live_viewer). E' idempotente, rispetta eventuali valori gia' impostati (usa la sintassi \${VAR:-default}) ed e' indipendente dalla macchina:

```
export ROS_DOMAIN_ID="${ROS_DOMAIN_ID:-0}"
export RMW_IMPLEMENTATION="${RMW_IMPLEMENTATION:-rmw_cyclonedds_cpp}"
if [ -z "${CYCLONEDDS_URI:-}" ] && [ -f "$HOME/.ros/cyclonedds.xml" ]; then
    export CYCLONEDDS_URI="file://$HOME/.ros/cyclonedds.xml"
fi
unset ROS_LOCALHOST_ONLY
```

Perche' cosi': il file cyclonedds.xml e' SPECIFICO della macchina (contiene l'interfaccia di rete e i peer, IP diversi tra laptop e server). Quindi ros_env.sh non lo sovrascrive mai: punta semplicemente a \$HOME/.ros/cyclonedds.xml e lo usa solo se esiste. Lo stesso script funziona su ogni macchina dopo un 'git pull'.

10.3 Dove viene agganciato

- panda_live_viewer/run.sh: il vecchio blocco di export inline e' stato sostituito da 'source "\$SCRIPT_DIR/ros_env.sh"' (DRY).
- lerobot/scripts/run_demo_bt_trial.sh: aggiunto 'source "\$REPO_ROOT/ros_env.sh"' subito dopo il calcolo di REPO_ROOT.
- ~/.bashrc (locale): aggiunto un blocco delimitato '>>> lerobot/panda ROS 2 + CycloneDDS env >>>' con gli stessi default, cosi' anche i terminali interattivi sono gia' configurati. Ho anche ripulito una riga malformata preesistente del blocco LEAP_Hand_tests.

10.4 Sul server

Gli script (run.sh, run_demo_bt_trial.sh) configurano l'ambiente in automatico anche sul server, perche' 'ros_env.sh' viaggia con il repo via git pull. Per i terminali interattivi sul server (dove non posso editare il suo .bashrc da qui) basta aggiungere UNA volta in fondo a ~/.bashrc:

```
source ~/lerobot/ros_env.sh          # oppure il percorso del repo sul server
```

Da quel momento ogni nuova shell sul server e' configurata, senza piu' export manuali.

11. Documentazione aggiornata e nota sull'ambiente Python

- docs/spatial_prior_gating.md: conteggio test 27->30; nuove sezioni 10 (calibrazione hand-eye / camera_static_tf_map), 11 (arricchimento scene-facts VLM one-way), 12 (smoke test offline).
- docs/DEMO_FINALE_COMMANDS.md: aggiornati i passi 11c.2 (smoke script + 30 test + avviso conda), 11c.5 (calibrazione hand-eye), 11b.8 (nota scene_facts del VLM).
- panda_live_viewer/README.md: note su arricchimento scene_facts in sola lettura,

camera_static_tf_map come fonte unica e helper di calibrazione.

Nota ambiente: gli script Python di questo lavoro vanno eseguiti con l'env conda 'lerobot' attivo (source di conda.sh + 'conda activate lerobot'). Il python nudo dell'env da' un errore CXXABI_1.3.15/libstdc++ nella catena di import (config.py -> transformers/scipy); l'attivazione conda sistema LD_LIBRARY_PATH.

12. Riepilogo file creati/modificati

12.1 Repo lerobot

- Modificato: src/lerobot_bt_python/spatial_prior_gate.py (parser dict+list).
- Modificato: src/lerobot_bt_python/test_spatial_prior.py (+3 test, 30 totali).
- Modificato: src/lerobot_bt_python/make_coffee_executor.yaml (placeholder camera_static_tf_map commentato).
- Nuovo: scripts/smoke_spatial_prior_gate.py (smoke test offline).
- Nuovo: ros_env.sh (env ROS 2/CycloneDDS centralizzato).
- Modificato: scripts/run_demo_bt_trial.sh (source ros_env.sh).
- Modificato: docs/spatial_prior_gating.md, docs/DEMO_FINALE_COMMANDS.md.
- Nuovo: scripts/genera_report_oggi.py (questo report).

12.2 Repo panda_live_viewer

- Nuovo: scripts/calibrate_camera_extrinsics.py (Kabsch/Umeyama).
- Modificato: vlm_live/prompt.py (format_scene_context, build_prompt).
- Modificato: vlm_live/node.py (iniezione scene_context one-way).
- Modificato: perception/node.py (rimosso TF estrinseco ridondante).
- Modificato: tests/test_vlm_status_protocol.py (+5 test, 15 totali).
- Modificato: README.md.
- Nuovo: ros_env.sh (env ROS 2/CycloneDDS centralizzato).
- Modificato: run.sh (source ros_env.sh).

13. Stato dei test

lerobot/test_spatial_prior.py: 30 test PASS. panda_live_viewer/test_vlm_status_protocol.py: 15 test PASS. smoke_spatial_prior_gate.py: PASS/FAIL/ABSTAIN tutti corretti. calibrate_camera_extrinsics.py: solver verificato con RMS=0 su trasformazione rigida esatta.

Falle preesistenti (NON causate da questo lavoro, confermate su HEAD pulito via git stash): test_perception_segmenters.py (test_labels_from_registry_uses_canonical_object_names) e test_vlm_node_static.py (2 test). Vanno trattate separatamente.